

Python Control Flow Quick Reference

A structured guide to Loops, Iteration Patterns, and Match-Case Statements

1. FOR LOOPS & ITERATION

```
# Iterating a specific number of times (0 to 4)  
for i in range(5):  
    print(i)
```

```
# Specifying a custom start value (1 to 5)  
for i in range(1, 6):  
    print(i)
```

```
# Using a step value (0, 2, 4, 6, 8)  
for i in range(0, 10, 2):  
    print(i)
```

```
# Loop through a string  
for ch in "Mazen":  
    print(ch)
```

```
# Loop through a list  
nums = [10, 20, 30]  
for num in nums:  
    print(num)
```

2. WHILE LOOPS

```
# Executes as long as the condition remains True  
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

3. LOOP CONTROL (BREAK & CONTINUE)

```
# BREAK: Exits loop completely  
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

```
# CONTINUE: Skips current iteration  
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

4. INFINITE LOOPS & SAFE EXITS

```
# Dangerous Infinite Loop  
while True:  
    print("Hello")
```

```
# Controlled Exit Scenario  
while True:  
    text = input("Enter q to quit: ")  
    if text == "q":  
        break
```

5. ADVANCED LOOP STRUCTURE & UTILITIES

```
# Nested Loops (Outer / Inner matrix interaction)  
for i in range(3):  
    for j in range(3):  
        print(i, j)
```

```
# Loop with Else (Executes only if loop completes without hitting a 'break')  
for i in range(5):  
    print(i)  
else:  
    print("Loop finished naturally")
```

```
# Enumerate (Track both sequence index and value simultaneously)  
names = ["Ali", "Ahmed", "Mazen"]  
for index, name in enumerate(names):  
    print(index, name)
```

6. COMMON RANGE PATTERNS SUMMARY

Range Function Anatomy

`range(n)` → 0 to n-1 | `range(a, b)` → a to b-1 | `range(a, b, step)` → a to b-1 incremented/decremented by step

```
# Forward sequence from 0 to 9
for i in range(10): pass

# Forward sequence from 1 to 10
for i in range(1, 11): pass

# Reverse sequence from 10 down to 1
for i in range(10, 0, -1): pass
```

7. STRUCTURAL PATTERN MATCHING (MATCH-CASE)

```
# Structural pattern matching syntax introduced in Python 3.10+
match value:
    case option1:
        statements
    case option2:
        statements
    case _:
        # Wildcard pattern acts as a default/catch-all case
        statements
```